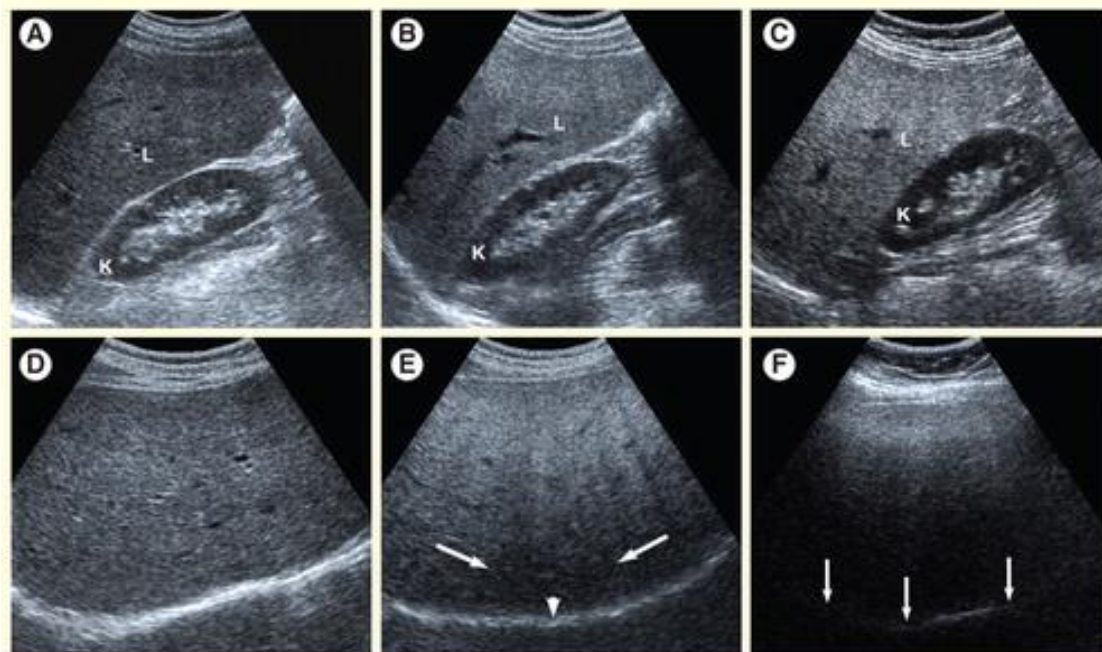


Liver Ultrasound Image Segmentation using U-net



```
import numpy as np
import pandas as pd
import os

base_dir = "/kaggle/input/annotated-ultrasound-liver-images-dataset/7272660"

categories = ["Benign", "Malignant", "Normal"]

data = []

for category in categories:
    image_dir = os.path.join(base_dir, category, category, "image")
    seg_dir = os.path.join(base_dir, category, category, "segmentation",
                           "liver")

    image_files = sorted([f for f in os.listdir(image_dir) if
                           f.lower().endswith(('.png', '.jpg', '.jpeg'))])
    json_files = sorted([f for f in os.listdir(seg_dir) if
                          f.lower().endswith('.json')])

    for img, jsn in zip(image_files, json_files):
        data.append({
            "Category": category,
            "Image_Path": os.path.join(image_dir, img),
            "Segmentation_JSON_Path": os.path.join(seg_dir, jsn)
```

```

    })

df = pd.DataFrame(data)

df.head()

  Category      Image_Path \
0  Benign  /kaggle/input/annotated-ultrasound-liver-image...
1  Benign  /kaggle/input/annotated-ultrasound-liver-image...
2  Benign  /kaggle/input/annotated-ultrasound-liver-image...
3  Benign  /kaggle/input/annotated-ultrasound-liver-image...
4  Benign  /kaggle/input/annotated-ultrasound-liver-image...

      Segmentation_JSON_Path
0  /kaggle/input/annotated-ultrasound-liver-image...
1  /kaggle/input/annotated-ultrasound-liver-image...
2  /kaggle/input/annotated-ultrasound-liver-image...
3  /kaggle/input/annotated-ultrasound-liver-image...
4  /kaggle/input/annotated-ultrasound-liver-image...

df.tail()

  Category      Image_Path \
727  Normal  /kaggle/input/annotated-ultrasound-liver-image...
728  Normal  /kaggle/input/annotated-ultrasound-liver-image...
729  Normal  /kaggle/input/annotated-ultrasound-liver-image...
730  Normal  /kaggle/input/annotated-ultrasound-liver-image...
731  Normal  /kaggle/input/annotated-ultrasound-liver-image...

      Segmentation_JSON_Path
727  /kaggle/input/annotated-ultrasound-liver-image...
728  /kaggle/input/annotated-ultrasound-liver-image...
729  /kaggle/input/annotated-ultrasound-liver-image...
730  /kaggle/input/annotated-ultrasound-liver-image...
731  /kaggle/input/annotated-ultrasound-liver-image...

df.shape

(732, 3)

df.columns

Index(['Category', 'Image_Path', 'Segmentation_JSON_Path'], dtype='object')

df.duplicated().sum()

0

df.isnull().sum()

Category      0
Image_Path    0

```

```
Segmentation_JSON_Path    0
dtype: int64
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 732 entries, 0 to 731
```

```
Data columns (total 3 columns):
```

#	Column	Non-Null Count	Dtype
0	Category	732 non-null	object
1	Image_Path	732 non-null	object
2	Segmentation_JSON_Path	732 non-null	object

```
dtypes: object(3)
```

```
memory usage: 17.3+ KB
```

```
import cv2
```

```
import json
```

```
import matplotlib.pyplot as plt
```

```
def generate_mask_from_json(json_path, image_shape):
    with open(json_path, 'r') as f:
        data = json.load(f)
```

```
    mask = np.zeros(image_shape[:2], dtype=np.uint8)
```

```
    def process_points(points):
        points = np.array(points, dtype=np.int32)
        if points.ndim == 2:
            return points
        elif points.ndim == 3:
            return points[0]
        else:
            return None
```

```
    if isinstance(data, list):
        for item in data:
            if isinstance(item, dict) and "points" in item:
                points = process_points(item["points"])
            elif isinstance(item, list):
                points = process_points(item)
            else:
                continue
```

```
        if points is not None and len(points) >= 3:
            cv2.fillPoly(mask, [points], 255)
```

```
    elif isinstance(data, dict) and "shapes" in data:
        for shape in data["shapes"]:
            points = process_points(shape["points"])
```

```

        if points is not None and len(points) >= 3:
            cv2.fillPoly(mask, [points], 255)

    return mask

def display_images_with_masks(df):
    categories = df['Category'].unique()
    fig, axs = plt.subplots(len(categories), 5, figsize=(20, 10))
    fig.suptitle("5 Random Liver Ultrasound Images per Category with Masks",
    fontsize=20)

    for row_idx, category in enumerate(categories):

        samples = df[df['Category'] == category].sample(5,
        random_state=42).reset_index(drop=True)

        for col_idx, row in samples.iterrows():
            img = cv2.imread(row['Image_Path'])
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

            try:
                mask = generate_mask_from_json(row['Segmentation_JSON_Path'],
                img.shape)
            except Exception as e:
                print(f"Error reading mask for
                {row['Segmentation_JSON_Path']}: {e}")
                mask = np.zeros(img.shape[:2], dtype=np.uint8)

            overlay = img.copy()
            overlay[mask == 255] = [255, 0, 0]

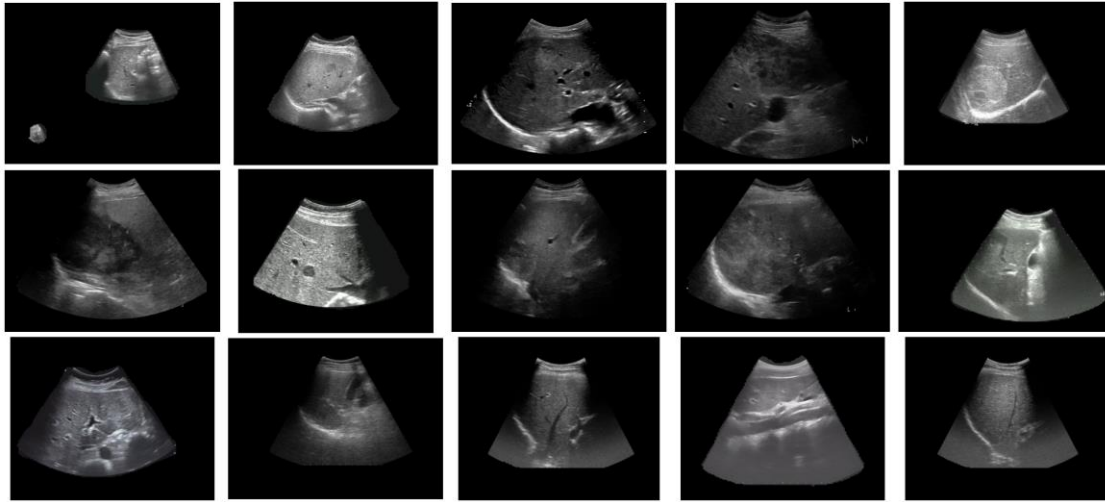
            axs[row_idx, col_idx].imshow(overlay)
            axs[row_idx, col_idx].axis('off')
            if col_idx == 0:
                axs[row_idx, col_idx].set_ylabel(category, fontsize=14)

    plt.tight_layout()
    plt.subplots_adjust(top=0.9)
    plt.show()

display_images_with_masks(df)

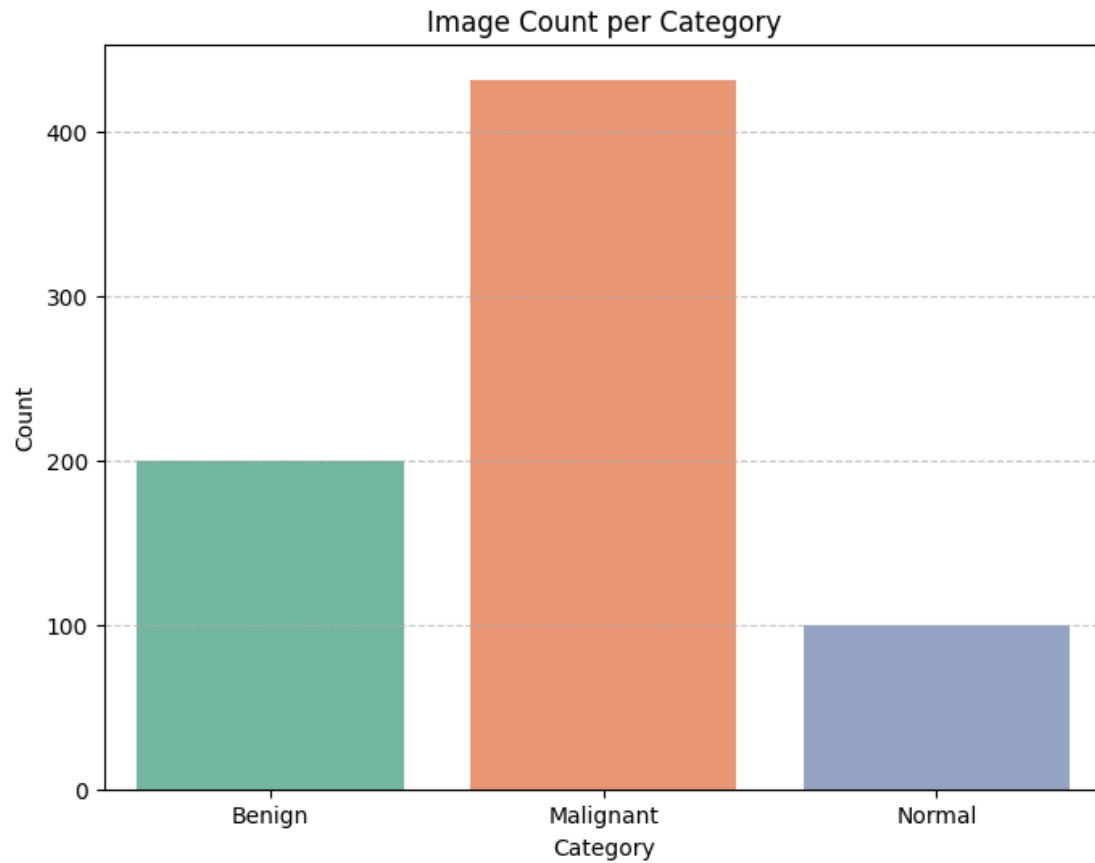
```

5 Random Liver Ultrasound Images per Category with Masks



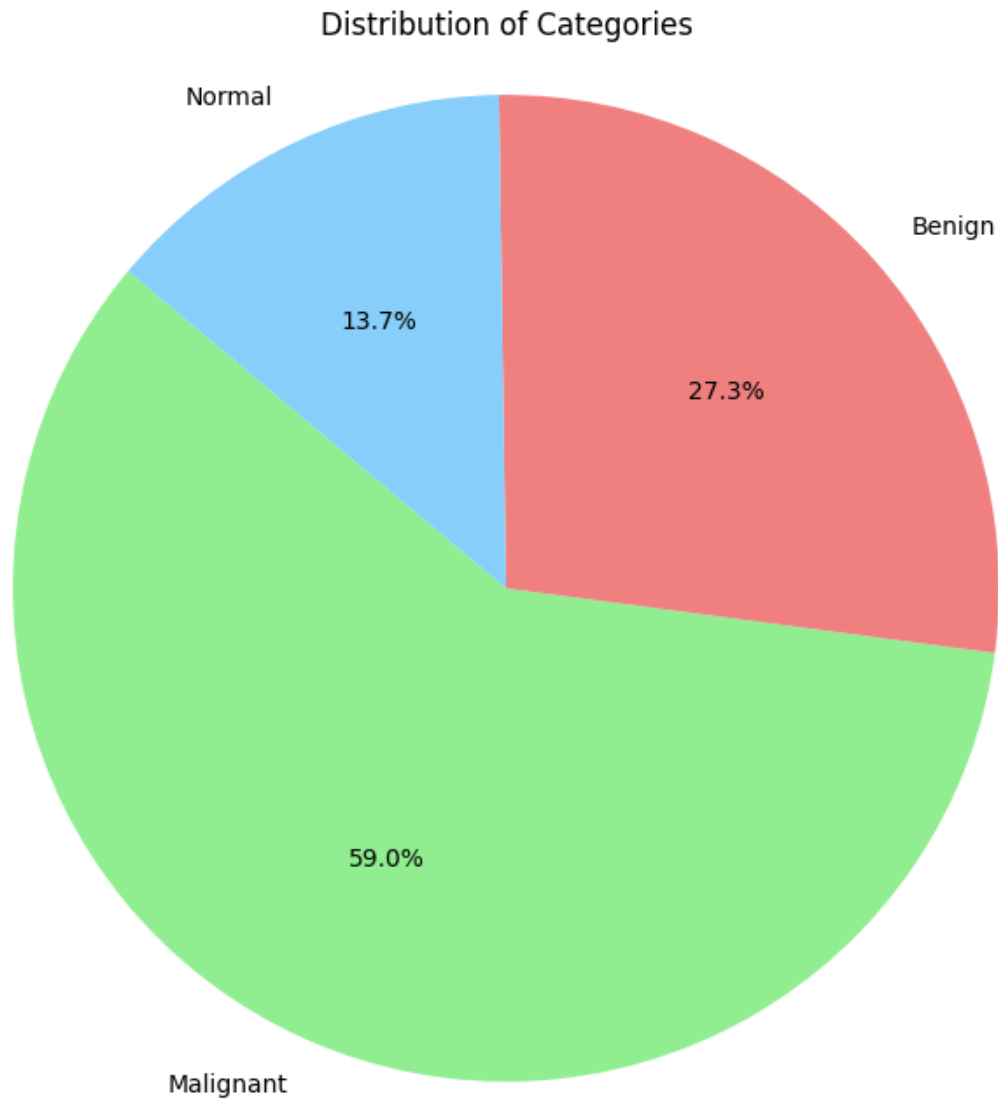
```
import seaborn as sns

plt.figure(figsize=(8, 6))
sns.countplot(x='Category', data=df, palette='Set2')
plt.title('Image Count per Category')
plt.xlabel('Category')
plt.ylabel('Count')
plt.grid(axis='y', linestyle='--', alpha=0.7)
plt.show()
```



```
category_counts = df['Category'].value_counts()
```

```
plt.figure(figsize=(8, 8))
plt.pie(category_counts, labels=category_counts.index, autopct='%1.1f%%',
startangle=140, colors=['lightgreen', 'lightcoral', 'lightskyblue'])
plt.title('Distribution of Categories')
plt.axis('equal')
plt.show()
```



```
from imblearn.over_sampling import RandomOverSampler
import pandas as pd

X = df.drop(columns=['Category'])
y = df['Category']

ros = RandomOverSampler(random_state=42)
X_resampled, y_resampled = ros.fit_resample(X, y)

df_balanced = pd.concat([X_resampled, y_resampled], axis=1)

print(df_balanced['Category'].value_counts())
```

Category
Benign 432

```
Malignant      432
Normal         432
Name: count, dtype: int64
```

```
import os
import cv2
import numpy as np
import json
import tensorflow as tf
from sklearn.model_selection import train_test_split
```

```
def generate_mask_from_json(json_path, image_shape):
    with open(json_path, 'r') as file:
        data = json.load(file)

    mask = np.zeros(image_shape[:2], dtype=np.uint8)

    if isinstance(data, list):
        for obj in data:
            if isinstance(obj, dict) and "points" in obj:
                points = np.array(obj["points"], dtype=np.int32)
                cv2.fillPoly(mask, [points], 255)
            elif isinstance(data, dict) and "shapes" in data:
                for shape in data["shapes"]:
                    points = np.array(shape["points"], dtype=np.int32)
                    cv2.fillPoly(mask, [points], 255)
    return mask
```

```
IMG_SIZE = 128
```

```
images, masks = [], []
for i, row in df_balanced.iterrows():
    img = cv2.imread(row['Image_Path'])
    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

    mask = generate_mask_from_json(row['Segmentation_JSON_Path'], img.shape)
    mask = cv2.resize(mask, (IMG_SIZE, IMG_SIZE))
    mask = np.expand_dims(mask, axis=-1)

    images.append(img / 255.0)
    masks.append(mask / 255.0)
```

```
images = np.array(images, dtype=np.float32)
masks = np.array(masks, dtype=np.float32)
```

```
X_train, X_val, y_train, y_val = train_test_split(images, masks,
test_size=0.2, random_state=42)
```



```

from tensorflow.keras import layers, models

def unet(input_shape=(128, 128, 3)):
    inputs = layers.Input(shape=input_shape)

    # Encoder
    c1 = layers.Conv2D(16, 3, activation='relu', padding='same')(inputs)
    c1 = layers.Conv2D(16, 3, activation='relu', padding='same')(c1)
    p1 = layers.MaxPooling2D()(c1)

    c2 = layers.Conv2D(32, 3, activation='relu', padding='same')(p1)
    c2 = layers.Conv2D(32, 3, activation='relu', padding='same')(c2)
    p2 = layers.MaxPooling2D()(c2)

    c3 = layers.Conv2D(64, 3, activation='relu', padding='same')(p2)
    c3 = layers.Conv2D(64, 3, activation='relu', padding='same')(c3)

    # Decoder
    u1 = layers.UpSampling2D()(c3)
    u1 = layers.concatenate([u1, c1])
    c4 = layers.Conv2D(32, 3, activation='relu', padding='same')(u1)
    c4 = layers.Conv2D(32, 3, activation='relu', padding='same')(c4)

    u2 = layers.UpSampling2D()(c4)
    u2 = layers.concatenate([u2, c1])
    c5 = layers.Conv2D(16, 3, activation='relu', padding='same')(u2)
    c5 = layers.Conv2D(16, 3, activation='relu', padding='same')(c5)

    outputs = layers.Conv2D(1, 1, activation='sigmoid')(c5)

    model = models.Model(inputs, outputs)
    return model

model = unet()
model.compile(optimizer='adam', loss='binary_crossentropy',
metrics=['accuracy'])
model.summary()

```

Model: "functional"

Layer (type)	Output Shape	Param #
Connected to		
input_layer (InputLayer)	(None, 128, 128, 3)	0 -

conv2d (Conv2D) input_layer[0][0]	(None, 128, 128, 16)	448
conv2d_1 (Conv2D) conv2d[0][0]	(None, 128, 128, 16)	2,320
max_pooling2d conv2d_1[0][0] (MaxPooling2D)	(None, 64, 64, 16)	0
conv2d_2 (Conv2D) max_pooling2d[0][0]	(None, 64, 64, 32)	4,640
conv2d_3 (Conv2D) conv2d_2[0][0]	(None, 64, 64, 32)	9,248
max_pooling2d_1 conv2d_3[0][0] (MaxPooling2D)	(None, 32, 32, 32)	0
conv2d_4 (Conv2D) max_pooling2d_1[0][0]	(None, 32, 32, 64)	18,496
conv2d_5 (Conv2D) conv2d_4[0][0]	(None, 32, 32, 64)	36,928
up_sampling2d conv2d_5[0][0] (UpSampling2D)	(None, 64, 64, 64)	0
concatenate (Concatenate) up_sampling2d[0][0], conv2d_3[0][0]	(None, 64, 64, 96)	0
conv2d_6 (Conv2D)	(None, 64, 64, 32)	27,680

concatenate[0][0]			
conv2d_7 (Conv2D)	(None, 64, 64, 32)	9,248	
conv2d_6[0][0]			
up_sampling2d_1	(None, 128, 128, 32)	0	
conv2d_7[0][0]			
(UpSampling2D)			
concatenate_1	(None, 128, 128, 48)	0	
up_sampling2d_1[0][0],			
(Concatenate)			
conv2d_1[0][0]			
conv2d_8 (Conv2D)	(None, 128, 128, 16)	6,928	
concatenate_1[0][0]			
conv2d_9 (Conv2D)	(None, 128, 128, 16)	2,320	
conv2d_8[0][0]			
conv2d_10 (Conv2D)	(None, 128, 128, 1)	17	
conv2d_9[0][0]			

Total params: 118,273 (462.00 KB)

Trainable params: 118,273 (462.00 KB)

Non-trainable params: 0 (0.00 B)

```
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=5,
    batch_size=16
)
```

Epoch 1/5

65/65 ————— 24s 166ms/step - accuracy: 0.9763 - loss: 0.2148 -
val_accuracy: 1.0000 - val_loss: 1.7607e-29

Epoch 2/5

65/65 ————— 2s 28ms/step - accuracy: 1.0000 - loss: 8.5957e-30
- val_accuracy: 1.0000 - val_loss: 4.6123e-30

```
Epoch 3/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 - loss: 4.6071e-30
- val_accuracy: 1.0000 - val_loss: 4.6049e-30
Epoch 4/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 - loss: 4.6049e-30
- val_accuracy: 1.0000 - val_loss: 4.6049e-30
Epoch 5/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 - loss: 4.6049e-30
- val_accuracy: 1.0000 - val_loss: 4.6049e-30
```

```
preds = model.predict(X_val[:5])

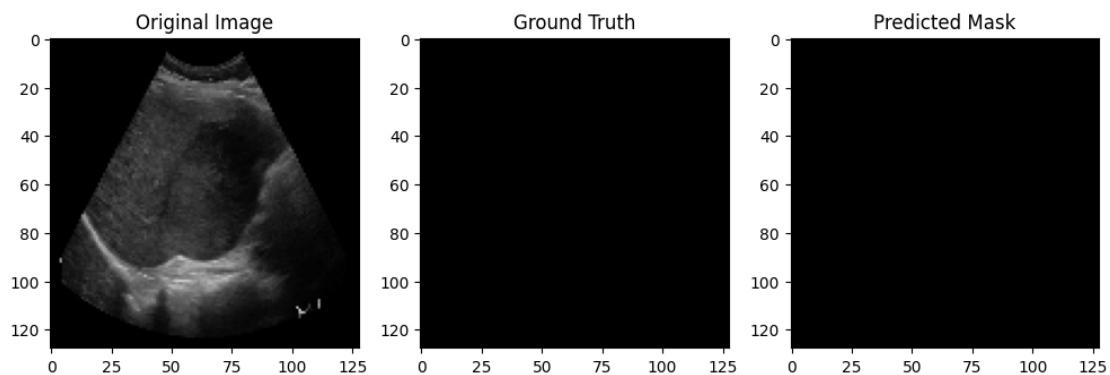
for i in range(5):
    plt.figure(figsize=(12, 4))

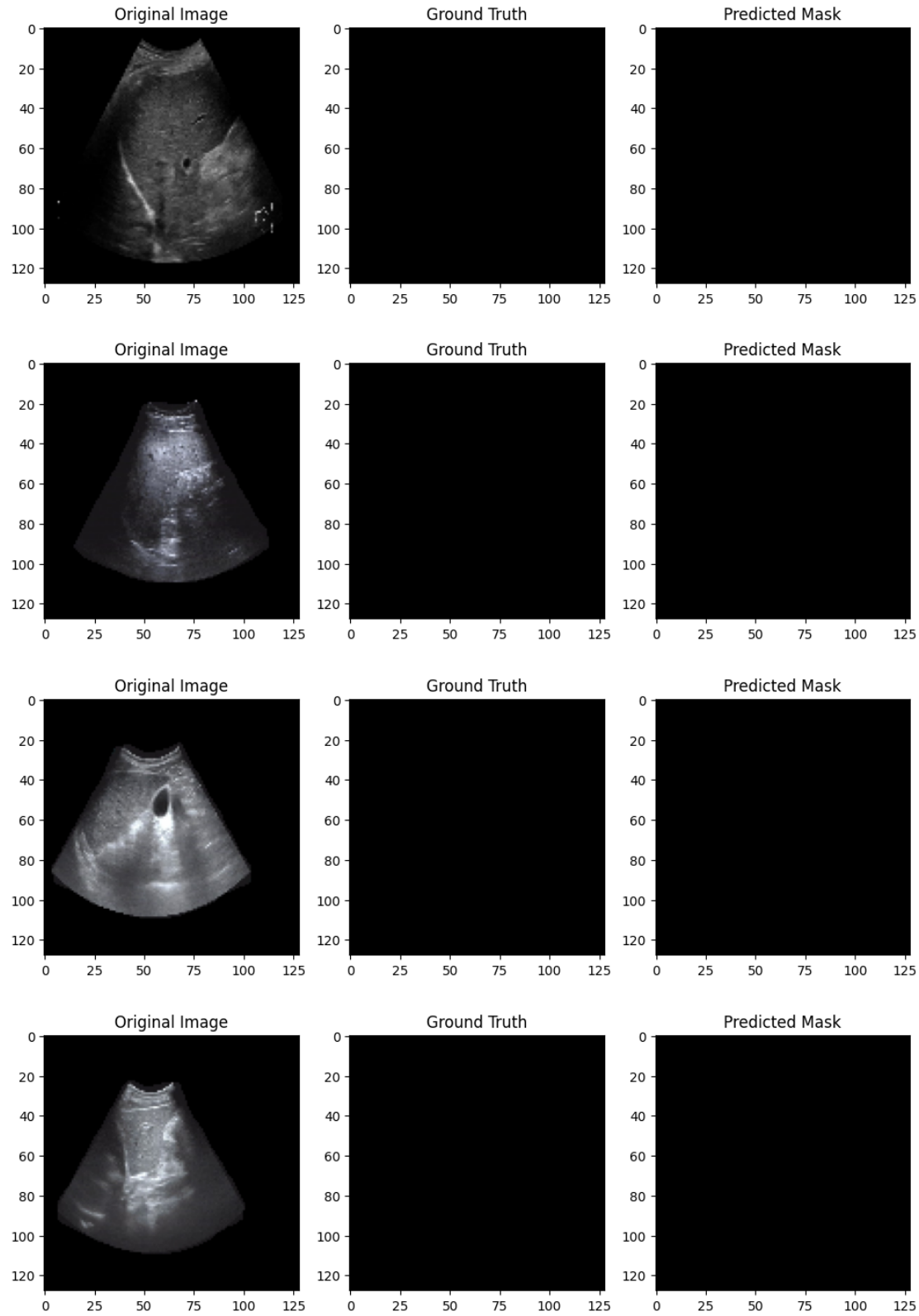
    plt.subplot(1, 3, 1)
    plt.title("Original Image")
    plt.imshow(X_val[i])

    plt.subplot(1, 3, 2)
    plt.title("Ground Truth")
    plt.imshow(y_val[i].squeeze(), cmap='gray')

    plt.subplot(1, 3, 3)
    plt.title("Predicted Mask")
    plt.imshow(preds[i].squeeze() > 0.5, cmap='gray')
    plt.show()
```

1/1 ————— 1s 1s/step





```
plt.figure(figsize=(12, 5))
```

```

plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', marker='o')
plt.title('Model Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

```

```

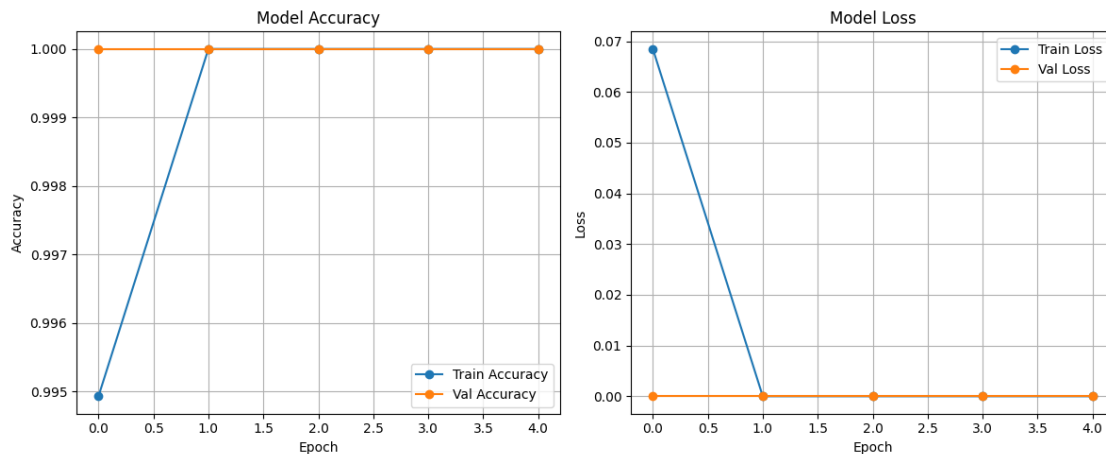
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Train Loss', marker='o')
plt.plot(history.history['val_loss'], label='Val Loss', marker='o')
plt.title('Model Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

```

```

plt.tight_layout()
plt.show()

```



```

import tensorflow as tf
from tensorflow.keras import backend as K

def dice_coefficient(y_true, y_pred, smooth=1e-6):
    y_true_f = K.flatten(y_true)
    y_pred_f = K.flatten(y_pred)
    intersection = K.sum(y_true_f * y_pred_f)
    return (2. * intersection + smooth) / (K.sum(y_true_f) + K.sum(y_pred_f) + smooth)

def dice_loss(y_true, y_pred):
    return 1 - dice_coefficient(y_true, y_pred)

```

```

model.compile(optimizer='adam',
              loss=dice_loss,
              metrics=['accuracy', dice_coefficient])

history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=5,
    batch_size=16
)

Epoch 1/5
65/65 ————— 9s 76ms/step - accuracy: 1.0000 -
dice_coefficient: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 -
val_dice_coefficient: 1.0000 - val_loss: 0.0000e+00
Epoch 2/5
65/65 ————— 2s 30ms/step - accuracy: 1.0000 -
dice_coefficient: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 -
val_dice_coefficient: 1.0000 - val_loss: 0.0000e+00
Epoch 3/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 -
dice_coefficient: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 -
val_dice_coefficient: 1.0000 - val_loss: 0.0000e+00
Epoch 4/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 -
dice_coefficient: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 -
val_dice_coefficient: 1.0000 - val_loss: 0.0000e+00
Epoch 5/5
65/65 ————— 2s 28ms/step - accuracy: 1.0000 -
dice_coefficient: 1.0000 - loss: 0.0000e+00 - val_accuracy: 1.0000 -
val_dice_coefficient: 1.0000 - val_loss: 0.0000e+00

plt.figure(figsize=(18, 5))

plt.subplot(1, 3, 1)
plt.plot(history.history['accuracy'], label='Train Accuracy', marker='o')
plt.plot(history.history['val_accuracy'], label='Val Accuracy', marker='o')
plt.title('Accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.subplot(1, 3, 2)
plt.plot(history.history['loss'], label='Train Dice Loss', marker='o')
plt.plot(history.history['val_loss'], label='Val Dice Loss', marker='o')
plt.title('Dice Loss')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()

```

```
plt.grid(True)

plt.subplot(1, 3, 3)
plt.plot(history.history['dice_coefficient'], label='Train Dice Coeff.',
marker='o')
plt.plot(history.history['val_dice_coefficient'], label='Val Dice Coeff.',
marker='o')
plt.title('Dice Coefficient')
plt.xlabel('Epoch')
plt.ylabel('Dice Coefficient')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

